Adaptive UAV-Aided Fair Task Offloading With Fuzzy Caching for Vehicular Edge Computing via Hierarchical Deep Reinforcement Learning

Qin Yang, *Graduate Student Member, IEEE*, and Sang-Jo Yoo, *Member, IEEE*

***Abstract*—Vehicular edge computing (VEC) faces critical challenges due to overloaded ground networks, high traffic demand, and limited resources, resulting in delayed responses and task failures. To address these issues, we propose an adaptive ground-air collaboration that integrates roadside unit (RSU) and unmanned aerial vehicles (UAVs) networks. Unlike existing methods with a fixed UAV configuration, our system dynamically learns when to deploy or retract UAVs based on real-time conditions, thereby improving aerial resource utilization. We introduce UDMP-CTOP, a two-timescale hierarchical framework based on proximal policy optimization (PPO), which consists of UAV deployment management (UDMP) and collaborative task offloading (CTOP). A multi-objective utility function is designed to minimize system costs while ensuring regional fairness, preventing localized performance degradation during global optimization. In addition to learning optimal policies for task offloading and cache placement, we address the underexplored cache replacement problem under edge memory constraints. To this end, we propose a fuzzy logic-based cache replacement (FCR) mechanism that considers data recency, frequency, and volume. Extensive simulations demonstrate that the proposed algorithms achieve superior task success rates and low system costs under diverse conditions, yielding higher total utility across various topologies while reducing UAV energy consumption.**

***Index Terms*—Cache placement and replacement, fuzzy logic, proximal policy optimization, roadside units, task offloading, unmanned aerial vehicles, vehicular edge computing.**

I. Introduction

THE rise of mobile applications, such as automatic navigation, infrastructure monitoring, and online games, has increased computation-intensive and delay-sensitive tasks, particularly in the Internet of Things (IoT) [1]. Mobile edge computing (MEC) addresses these challenges by bringing computation and storage closer to end-users. Within the IoT ecosystem, the Internet of Vehicles (IoV) connects vehicles via the Internet to facilitate real-time communication [2]. This connectivity enhances traffic management and safety, essential for vehicular applications like autonomous driving requiring low-latency communication [3]. Vehicular edge computing (VEC), merging MEC principles with IoV, supports applications ranging from collision avoidance to infotainment [4]. However, VEC faces challenges regarding dynamic topology and resource limitations. Thus, efficient task offloading and caching strategies in MEC or VEC are crucial for improving service quality [5].

Current edge computing approaches using roadside units (RSUs) or base stations (BSs) are widely adopted. Guo et al. [6] introduced fiber-wireless integration to ensure quality-of-service (QoS), supporting the coexistence of remote cloud and edge servers with vehicle-to-everything (V2X). Yang et al. [7] explored infrastructure-based systems to manage data routing within ground-based networks. Nevertheless, their static nature often fails to address dynamic scenarios, such as sudden traffic surges, necessitating flexible solutions for reliable vehicular edge computing.

Unmanned aerial vehicles (UAVs) are incorporated into IoT networks to enhance adaptability. UAVs complement static RSUs, providing on-demand computational and relay capabilities, with LTE enabling high-speed transmission. Beishenalieva et al. [8] optimized UAV trajectory in wireless networks using particle swarm optimization (PSO) to maximize utility. Liao et al. [9] presented a PSO-based algorithm to deploy UAV clusters for continuous services. Kuang et al. [10] combined UAV deployment, offloading, and resource allocation, ensuring task success rates under energy constraints. Typically, deploying a fixed number of UAVs poses challenges due to dynamic traffic demands. UAVs may remain idle at times, resulting in resource wastage and energy inefficiency.

Edge caching for IoT faces unique challenges unlike traditional systems. Zhang et al. [11] examined a caching strategy considering data lifetime to minimize energy consumption. Zhu et al. [12] proposed an improved non-dominated sorting genetic algorithm for vehicle-cloud collaborative caching. Liu et al. [13] outlined a joint hybrid caching scheme where a single UAV assists the RSU to minimize task delay.

Received 18 June 2025; revised 2 October 2025; accepted 21 December 2025. Date of publication 24 December 2025; date of current version 24 June 2026. This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation-Information Technology Research Center funded by the Korea Government (Ministry of Science and ICT) under Grant RS-2021-II212052 and in part by the National Research Foundation of Korea funded by the Korea Government (MSIT) under Grant RS-2025-21212972. The review of this article was coordinated by Dr. Phone Lin. *(Corresponding author: Sang-Jo Yoo.)*

The authors are with the Department of Electrical and Computer Engineering, Inha University, Incheon 22212, South Korea (e-mail: sjyoo@inha.ac.kr).

Digital Object Identifier 10.1109/TVT.2025.3647844

0018-9545 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

Artificial intelligence (AI) techniques, specifically deep reinforcement learning (DRL), are used to manage the complexities and unpredictability of such hybrid networks [14], [15], [16]. Despite recent advances, existing works still exhibit three major limitations. First, most task offloading studies focus on maximizing global performance while neglecting regional service fairness, resulting in persistent imbalance across edge servers. Second, cache replacement strategies have been insufficiently explored in limited-memory edge caching environments. Third, UAV-assisted systems often assume static configurations, which can cause over-provisioning or under-provisioning of aerial resources in dynamic vehicular scenarios. These limitations motivate the design of a hierarchical proximal policy optimization (PPO) framework that integrates fairness-aware task offloading, a fuzzy logic–based cache replacement mechanism, and adaptive UAV deployment, thereby addressing the key challenges in scalable and efficient vehicular edge computing. The main contributions of this study are as follows:

- *Adaptive UAV Deployment for Ground-Air Collaboration:* We propose a novel adaptive ground-air collaboration system where UAV deployment and retraction are dynamically managed in real time based on vehicular traffic load and network conditions. Unlike prior works that rely on static UAV configurations, our approach optimizes UAV energy consumption and ensures responsiveness to ground demand fluctuations.
- *Two-Timescale Hierarchical PPO Framework:* We develop UDMP-CTOP, a hierarchical PPO framework operating across two timescales. The short-timescale collaborative task offloading (CTOP) makes fine-grained offloading and caching decisions, while the long-timescale UAV deployment management (UDMP) makes UAV activation and positioning decisions. Crucially, this enables mutual coordination between UDMP and CTOP: UDMP dynamically adjusts UAV deployment by utilizing insights from CTOP, while CTOP optimizes task offloading and caching based on the current UAV positions determined by UDMP. This cross-layer interaction ensures consistent adaptation across timescales in VEC environments.
- *Fairness-Aware Multi-Objective Utility Function:* We design a multi-objective utility function that jointly optimizes task delay, energy consumption, task success rate, and regional fairness. This formulation ensures balanced service delivery and prevents performance degradation in specific regions during global optimization.
- *Fuzzy Logic-Based Cache Replacement Mechanism (FCR):* To address the underexplored challenge of cache replacement under edge memory limits, we introduce a fuzzy logic-based mechanism. FCR identifies replaceable content using recency, frequency, and volume metrics. Upon exceeding capacity thresholds, it swaps low-priority data to optimize availability and efficiency.

The remainder of this paper is structured as follows. Section II introduces the related work. Section III elaborates on the network architecture and describes the system models. Section IV defines the problem formulation. Section V outlines the proposed hierarchical PPO algorithm for fair task offloading and adaptive UAV deployment. Section VI introduces the FCR mechanism for cache replacement. The simulation results are presented and discussed in Section VII. Finally, this paper is concluded in Section VIII.

II. Related Work

A. Task Offloading Using Heuristics and Optimization

Heuristic and optimization techniques have been widely applied to task offloading in edge computing environments. In RSU-based schemes, RSUs process tasks offloaded from vehicles. Dai et al. [17] combined load balancing and offloading in VEC networks using heuristic algorithms to minimize delay and maximize resource utilization. Hou et al. [18] developed a fault-tolerant PSO algorithm for latency-sensitive IoV applications, while Luo et al. [19] formulated computation offloading as a distributed game and proposed a self-learning algorithm to reach Nash equilibrium. Although heuristic approaches are efficient for complex optimization problems, they often converge to suboptimal solutions in highly dynamic environments.

In UAV-assisted offloading, UAVs manage computational workloads with an emphasis on mobility and energy control. Tun et al. [20] applied Lagrangian relaxation and the alternating direction method of multipliers (ADMM) for communication resource allocation in UAV-assisted MEC. Xu et al. [21] transformed a non-convex completion-time minimization problem into a convex form, while Zhan et al. [22] and Guo et al. [23] optimized UAV energy consumption and trajectories using alternating optimization. Liu et al. [24] employed successive convex approximation for joint trajectory and CPU control, and Dai et al. [25] applied Lyapunov optimization to address long-term UAV energy constraints. However, optimization-based approaches generally lack adaptability to rapidly changing vehicular environments. In contrast, DRL-based methods enable continuous learning and dynamic adaptation.

B. Task Offloading Using DRL

DRL has demonstrated strong capability in handling the complexity and uncertainty of task offloading in dynamic edge computing networks. Mirza et al. [26] proposed a DRL-based cooperative offloading scheme for IoV, incorporating mobility, contact, and load awareness. Deng et al. [27] integrated variational graph attention networks and transformer models for DAG task scheduling using DRL, while Xue et al. [28] and Wu et al. [29] developed RSU-assisted DRL frameworks for joint offloading and resource allocation to reduce delay and energy consumption.

In UAV-assisted MEC networks, Gao et al. [30] introduced a multi-agent RL framework for joint UAV trajectory planning and task offloading. Lee and Kim [31] optimized offloading ratios and UAV trajectories to improve energy efficiency and queue stability, while Zhao et al. [32] addressed collaborative UAV trajectory and resource management under a DRL framework. In UAV-assisted VEC scenarios, Li et al. [33] proposed a dueling double deep Q-network for latency minimization

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING TASK OFFLOADING WORKS

Work	Techniques	Task Routing	Caching		Aerial Layer		Fairness Awareness
			Placement	Replacement	UAV Assistance	Adaptive activation	
[20]	Optimization	×	×	×	✓	×	×
[46]	Heuristic	✓	×	×	×	×	✓
[39]	Heuristic+DRL	✓	✓	×	×	×	×
[14]	LSTM+DRL	×	×	×	✓	×	×
[40]	DRL	×	✓	×	×	×	×
[13]	DRL	×	✓	✓	✓	×	×
[36]	DRL	✓	✓	×	×	×	×
[51]	DRL	×	×	×	✓	×	✓
Ours	DRL	✓	✓	✓	✓	✓	✓

under UAV constraints, and Yan et al. [34] developed an SDN-driven DRL-based framework for joint trajectory control and offloading allocation. Despite these advances, most DRL-based approaches prioritize global performance optimization while overlooking regional fairness. Moreover, the assumption of a fixed number of UAVs limits scalability and may lead to energy inefficiency.

C. Joint Task Offloading and Caching

Joint optimization of task offloading and caching has been investigated to reduce latency by placing frequently accessed data closer to end users. Wang et al. [35] addressed computation offloading, resource allocation, and content caching in MEC-enabled cellular networks using an ADMM-based algorithm, while Wang et al. [36] applied a double deep Q-network to optimize service caching and offloading in a three-tier MEC architecture. Shen et al. [37] and Ning et al. [38] formulated mixed-integer non-linear programming (MINLP) models for joint offloading and caching in VEC networks, with the latter incorporating Lyapunov optimization. Liu et al. [39] integrated energy harvesting, sub-channel allocation, caching, and offloading using a hybrid simulated annealing and DRL approach, and Ghorab et al. [40] proposed a cluster-based DRL framework for microservice caching at RSUs.

However, most joint offloading–caching studies neglect the role of UAVs. Beishenalieva et al. [8] investigated asynchronous UAV trajectory optimization in three-dimensional environments, while an RL-based approach in [44] managed UAV trajectories using spatial–temporal states. Although UAV integration can improve flexibility and coverage, existing works primarily focus on cache placement, leaving cache replacement mechanisms underexplored in task offloading scenarios.

To provide a comprehensive comparison, representative studies are summarized in Table I, which contrasts heuristic/optimization-based methods, DRL-based offloading schemes, and joint offloading–caching approaches in terms of task routing, caching support, UAV involvement, and fairness considerations. In contrast, our proposed framework uniquely integrates hierarchical DRL with adaptive UAV activation, fairness-aware optimization, and fuzzy cache replacement to address key limitations of prior work.

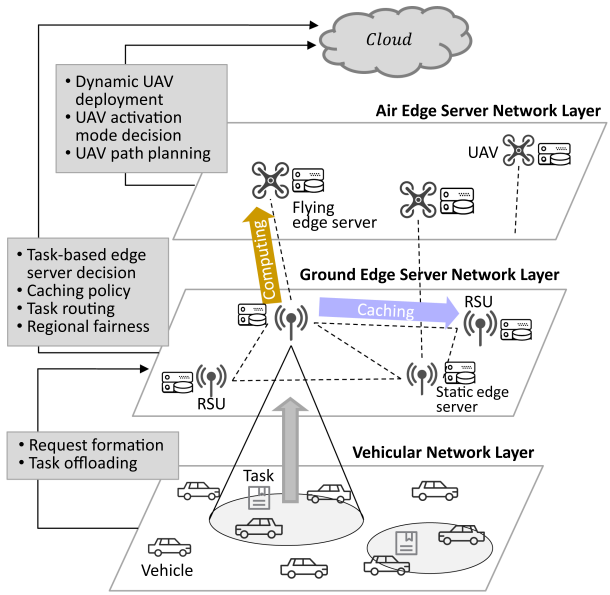


Fig. 1. The ground-air layered structure.

III. SYSTEM MODEL

A. Network Model

A ground-air layered structure is considered, as illustrated in Fig. 1. We define a vehicular network layer with a vehicle set $\mathcal{V} = \{\mathrm{V}_1, \mathrm{V}_2, \cdots, \mathrm{V}_{\mathrm{N_V}}\}$, where N_V is the total number of vehicles. The ground edge server network layer consists of RSUs $\mathcal{R} = \{\mathrm{R}_1, \mathrm{R}_2, \cdots, \mathrm{R}_{\mathrm{N_R}}\}$, where N_R is the total number of RSUs. RSUs receive requests from vehicles within their communication range. In the air edge server network layer, flying edge servers are defined as $\mathcal{U} = \{\mathrm{U}_1, \mathrm{U}_2, \cdots, \mathrm{U}_{\mathrm{N_U}}\}$, where N_U is the total number of UAVs. The total node set is $\mathcal{N} = \{\mathcal{V} \cup \mathcal{R} \cup \mathcal{U}\}$, and the edge server set is $\mathcal{M} = \{\mathcal{R} \cup \mathcal{U}\}$. A central cloud server at the top layer manages the ground and air network layers. Each edge server (ES) inherently operates under constrained computational power and caching capacity due to its edge nature. To reflect network heterogeneity, we consider two types: $ES1$, which supports both computation and caching but with limited resources, and $ES2$, which only provides caching capability without computation. The computation resource is

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MAIN NOTATION SUMMARY

Notation	Description	Notation	Description
$\mathcal{V}, N_V$	Set of vehicle nodes, number of vehicles	$g_{k\to m}$	Computing resource required on edge server m to process task k
$\mathcal{R}, N_R$	Set of RSUs , number of RSUs	e_m^{cp}	Computing energy of edge server m
$\mathcal{U}, N_U$	Set of UAVs, number of UAVs	z_k^O, z_k^D	Offloading size of task k, downloading size of task k
$\mathcal{N}, \mathcal{M}$	Set of all nodes, set of all edge servers	$\mathcal{U}_a, NU_a$	Set of activated UAVs, number of activated UAVs
$ES1$, $ES2$	Edge servers equipped with both caching and computing capabilities, Edge servers with only caching capability (no computing)	$act_{\mathcal{U}}$, $pos_{\mathcal{U}_a}$	Set of activation statuses of all UAVs, Set of positions of activated UAVs
$\mathcal{O}, O_m^{init}$	Set of computation resources of all edge servers, the initial resource of edge server m	$e_{i,j}^{com}$, e_k^{com}	Communication energy between any node i and j, communication energy along the path to process task k
$\mathcal{C}, C_m^{init}$	Set of caching capacity of all edge servers, the initial capacity of edge server m	e_u^{fly}, e_u^{hov}, E_u^{mov}	Flying energy of UAV u, hovering energy of UAV u, total movement energy of UAV u
TS, h	Time slot, h-th time slot	E_u, E_u^{max}	Total energy of UAV u, the maximum energy of UAV u
$\mathcal{T}(h)$, $K(h)$	Set of tasks at h-th time slot, number of tasks at h-th time slot	$a_{k\to m}^{ce}$	Cache placement indicator of task k on edge server m
P_k^h	Information vector of task k at h-th time slot	a_k^{dest}	Offloading decision of task k
$freq_k$	Frequency of task k	d_m^{cp}	Computing delay on edge server m
$r_{R2R}, r_{i,j}$	Wired transmission rate between RSUs, wireless transmission rate between any node i and j	D_k, D^{max}	Total delay to process task k, the normalized delay value
$d_{i,j}^{tx}, d_k^{tx}$	Transmission delay between any node i and j, transmission delay along the path to process task k	E_k, E^{max}	Total energy to process task k, the normalized energy value
d_k^{pg}	Propagation delay along the path to process task k	suc_k	Success indicator of task k
d_k^{que}	Queuing delay along the path to process task k	$F_{k,m}$	Regional fairness metric of task k on edge server m

expressed as $\mathcal{O} = \{\mathrm{O_m} \mid \mathrm{m} \in \mathcal{M}\}$, and $\mathrm{O_m} = 0$ for ES2. The caching capacity of all edge servers is $\mathcal{C} = \{\mathrm{C_m} \mid \mathrm{m} \in \mathcal{M}\}$. $\mathrm{O_m^{init}}$ and $\mathrm{C_m^{init}}$ represent the initial resources, respectively. The main notations are summarized in Table II.

B. Task Model

1) Task Offloading Model: This paper adopts a binary offloading strategy, where each task is fully offloaded and executed without partitioning due to security or precision constraints [41]. Time is divided into intervals, and each time slot (TS) corresponds to an aerial decision interval. Let h denote the h-th time slot, defined as $[(h-1)\times TS, h\times TS]$. The task set in slot $h$is $T(h) = \{T_1^h, T_2^h, \ldots, T_{K(h)}^h\}$, where $K(h)$ is the number of tasks. Each task is characterized by an information vector $P_k^h = \langle v_k^{info}, p_k^{info}\rangle$. The mobility component $v_k^{info} = \langle ID_v, loc_v, vel_v, dire_v\rangle$ captures vehicle information, while the task profile $p_k^{info} = \langle gt_k, tp_k, z_k^O, dt_k, freq_k\rangle$ includes the generation time, task type, input size, delay tolerance, and access frequency, which follows a Zipf distribution [11], [42], [43]. Tasks are classified into computation tasks (e.g., driving assessment and autonomous decisions) and content tasks (e.g., video, music, and maps).

As illustrated in Fig. 2, a ground vehicle generates tasks and transmits them to nearby RSUs via V2R communication. If local resources are insufficient, tasks are forwarded to neighboring RSUs through wired links or to UAVs via wireless connections. After processing, results are returned to the vehicle and cached at edge servers; a task is considered successful if completed within its delay tolerance. Cache replacement is triggered when storage exceeds a threshold, while UAVs are dynamically activated and repositioned according to ground demand. Each UAV is equipped with a CPU, cache, wireless module, and battery-powered mobility engine. Due to inherent wireless constraints such as mobility and non-line-of-sight conditions, V2R transmission may incur packet loss; therefore, it is treated as a pre-offloading stage. This work focuses on optimizing task offloading after tasks reach RSUs, where more reliable R2R and R2U links dominate, and investigates multi-hop and UAV-assisted offloading strategies to enhance system performance and promote fair task allocation in heterogeneous V2X environments.

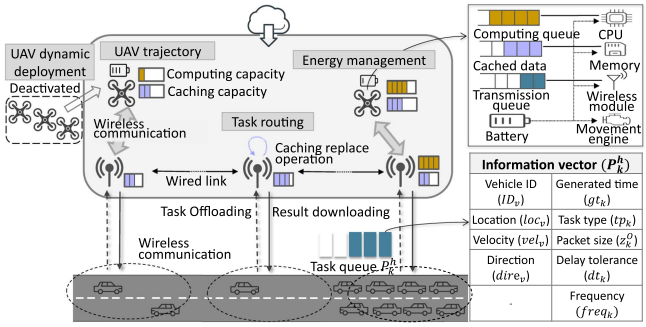


Fig. 2. Task offloading scheme.

2) Task Caching Model: The caching model can be effectively categorized into two primary operations: cache placement and replacement. Each operation is crucial for optimizing the utilization of limited storage resources. The cache placement decision is modeled using a binary indicator $a_{k\to m}^{ce} \in \{0,1\}$, where $a_{k\to m}^{ce} = 0$ signifies the data is not cached, and $a_{k\to m}^{ce} = 1$ indicates that the data is cached. This decision is influenced by factors such as the task's popularity $(freq_k)$ and the current storage status. The system evaluates these factors to determine whether the available storage can accommodate new data without compromising the caching performance. The replacement operation is triggered autonomously when the current storage status exceeds a predefined threshold C_0, such that $C_m/C_m^{init} \geq C_0$. This condition indicates that the storage capacity is strained, necessitating the activation of a cache replacement mechanism. The mechanism operates in real-time to remove the less critical cached data, freeing up space and ensuring the system maintains efficient storage.

C. Communication Model

The communication model is divided into wired and wireless channels. The communication model considers three link types:

- *RSU-to-RSU (R2R):* Assumed as a wired backhaul with constant capacity among RSUs, and the transmission rate is denoted as r_{R2R} [6], [34].
- *RSU-to-UAV (R2U):* Modeled as a wireless LTE/5G backhaul with channel gain and directional bandwidth (uplink/downlink). In R2U, a RSU is assumed to be associated with a dedicated UAV. R2U links are considered orthogonal and independently scheduled so that R2U link is generally more stable than V2R.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

- *GV-to-RSU (V2R):* Treated as part of the pre-offloading stage and compatible with CV2X, IEEE 802.11p, and NR-V2X. While packet loss may occur due to fading and mobility, its impact is largely confined to the pre-offloading stage so that it is assumed it does not directly affect the offloading optimization among RSUs and UAVs.

To ensure consistency, symbols i and j denote nodes sharing the wireless channel. The transmission rate between nodes i and j is

$$r_{i,j} = B\log_2\left(1+\frac{\boldsymbol{p}_i^{tx}(dist_{i,j})^{-\beta}\cdot|G_0|^2}{N_0}\right), i,j\in\mathcal{N}, i\neq j, \tag{1}$$

where $B\in\{B^{UL},B^{DL}\}$ is the uplink or downlink bandwidth, p_i^{tx} is the transmit power of node i, and N_0 denotes additive white Gaussian noise. The Euclidean distance is $dist_{i,j}=\sqrt[2]{(x_i-x_j)^2+(y_i-y_j)^2+(z_i-z_j)^2}$, with (x_i,y_i,z_i) and (x_j,y_j,z_j) being node coordinates. Here, β is the path-loss exponent and G_0 is the complex Gaussian channel coefficient. The transmission delay is computed as follows:

$$d_{i,j}^{tx}=\frac{z_k}{r_i^{tx}},\ z_k\in\left\{z_k^O,z_k^D\right\},\ r_i^{tx}\in\left\{r_{R2R},r_{i,j}\right\}, \tag{2}$$

where z_k^D denotes the size of the returned result or content; computation tasks typically generate smaller results than content tasks. Receiving energy is ignored. The communication energy consumption is

$$e_{i,j}^{com}=\frac{\boldsymbol{p}_i^{tx}z_k}{r_i^{tx}},\ z_k\in\left\{z_k^O,z_k^D\right\},\ r_i^{tx}\in\left\{r_{R2R},r_{i,j}\right\}. \tag{3}$$

D. Computing Model

The CPU of an edge server is the primary engine for processing computing tasks. Its performance is determined by the cycle frequency, commonly referred to as the CPU clock speed, which indicates the computation capability. The number of CPU cycles required for task k is expressed as

$$g_{k\to m}=z_k^O\ \rho,\ m\in\mathcal{M}, \tag{4}$$

where $g_{k\to m}$ is the computing resource required to process task k on edge server m, and ρ denotes the number of CPU cycles required by the edge server to calculate one bit of a task. The computing delay occurs at the final offloading decision edge and includes queuing and computing delay as follows:

$$d_m^{cp}=\frac{z_m^{que}\rho+g_{k\to m}}{f_m},\ m\in\mathcal{M}, \tag{5}$$

where z_m^{que} and f_m are the queuing size for computation and the CPU cycle frequency of the edge server, respectively. The computing energy consumption [5], [14] is calculated using

$$e_m^{cp}=\lambda f_m^3 d_m^{cp},\ m\in\mathcal{M}, \tag{6}$$

where λ is an effective capacitance coefficient.

E. UAV Movement Model

Each UAV is assigned a binary activation status $act_u\in\{0,1\}$, where $act_u=1$ indicates that UAV u is activated to assist ground tasks. The activation set of all UAVs is denoted by $act_U=\{act_1,\ldots,act_{N_U}\}$, and the subset of activated UAVs is $U_a\subseteq U$ with cardinality N_U^a. Each activated UAV can hover above any ground edge server and serve at most one RSU at a time. The position of UAV u is defined as $pos_u=\{[x_R,y_R,z_R]\mid R\in\mathcal{R}\}$, and the positions of all activated UAVs are denoted by pos_{U_a}. UAVs move at a constant speed $v_u$bounded by $v_u^{\max}$, yielding a maximum per-slot displacement constraint $\parallel pos_u(h)-pos_u(h-1)\parallel\leq dist_u^{\max}$. UAVs are assumed to maintain a fixed altitude and move only horizontally. The flying time is given by $t_u^{fly}=\parallel pos_u(h)-pos_u(h-1)\parallel/v_u$, and the hovering time for communication is denoted by t_u^{hov}.

To model UAV energy consumption [8], [44], we consider movement, communication, and computing costs. Movement energy consists of flying and hovering components:

$$e_u^{fly}=\boldsymbol{p}_u^{fly}\ t_u^{fly}, \tag{7}$$

$$e_u^{hov}=\boldsymbol{p}_u^{hov}\ t_u^{hov}, \tag{8}$$

where $\boldsymbol{p}_u^{fly}$ and $\boldsymbol{p}_u^{hov}$ are the unit energy costs for flying and hovering, respectively, resulting in total movement energy

$$E_u^{mov}=e_u^{fly}+e_u^{hov}. \tag{9}$$

For retracted UAVs, only flying energy is incurred. Accordingly, the total energy consumption of UAV u is

$$E_u=E_u^{mov}+e_{u,j\in\mathcal{R}}^{com}+e_u^{cp}, \tag{10}$$

where $e_{(u,j\in\mathcal{R})}^{com}$ denotes the communication energy with RSU j, and e_u^{cp} is the computing energy on UAV u.

IV. Problem Formulation

This study develops a multi-objective optimization that minimize system cost while maximizing the task success rates and ensuring the regional fairness. We define the offloading decision for each task k as

$$a_k^{dest}\in\{\mathcal{R}\cup\mathcal{U}_a\}, \tag{11}$$

indicating that a task can be offloaded either to a ground edge server or to an activated UAV.

The total delay cost to process task k is given by

$$D_k=d_k^{tx}+d_k^{pg}+d_k^{que}+d_k^{cp}. \tag{12}$$

The transmission delay is defined as

$$d_k^{tx}=\sum_{i,j\in PATH_k^{R_s\to M_e}}d_{i,j}^{tx}. \tag{13}$$

The propagation delay is computed by

$$d_k^{pg}=\sum_{i,j\in PATH_k^{R_s\to M_e}}\frac{dist_{i,j}}{v_{pg}}. \tag{14}$$

The queuing delay is given by

$$d_k^{que}=\sum_{i,j\in PATH_k^{R_s\to M_e}}\frac{z_j^{que}}{r_i^{tx}}. \tag{15}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

The computing delay at the final edge server M_e is expressed as

$$d_k^{cp} = d_{M_e}^{cp} = \frac{z_{M_e}^{que}\rho + g_{k\to M_e}}{f_{M_e}}. \tag{16}$$

The offloading path $PATH_k^{R_s\to M_e}$ is defined from the ingress RSU of task k to the selected execution server (a RSU or UAV). The initial GV-to-RSU transmission delay is considered part of the pre-offloading stage and is therefore excluded from the optimization. This assumption isolates the RSU-level offloading process, which is the focus of our design, and does not impact the generality of the results since tasks are assumed to be already available at their serving RSUs before offloading decisions are made. v_{pg} is the propagation speed and z_j^{que} denotes the queuing size for transmission at node j.

Similarly, the total energy cost to process task k includes communication and computing energy as

$$E_k = e_k^{com} + e_k^{cp}, \tag{17}$$

with

$$e_k^{com} = \sum_{i,j\in PATH_k^{R_s\to M_e}} e_{i,j}^{com}, \tag{18}$$

and

$$e_k^{cp} = e_{M_e}^{cp} = \lambda f_{M_e}^3 d_{M_e}^{cp}. \tag{19}$$

The task success indicator is defined as

$$suc_k \in \{0,1\}, \tag{20}$$

where $suc_k = 1$ if task k is successfully processed and delivered back to the generating vehicle; otherwise, $suc_k = 0$. In addition to overall system performance, we introduce the fairness metric $(F_{k,m})$ to evaluate regional fairness across edge servers. While minimizing delay and energy cost improves global performance, it may lead to persistent underperformance in certain regions. The fairness metric mitigates such disparities by quantifying the balance of service quality among different edges and ensuring that no region is systematically disadvantaged.

Thus, the muti-objective utility function is defined as

$$\mathbb{U}_k = \mathbb{E}\left[-\boldsymbol{w}_d\frac{D_k}{D^{\max}} - \boldsymbol{w}_e\frac{E_k}{E^{\max}} + \boldsymbol{w}_s suc_k + \boldsymbol{w}_f F_{k,m}\right], \tag{21}$$

where $D^{\max}$ and $E^{\max}$ are normalized values of delay and energy to ensure dimensional consistency, respectively. $F_{k,m}$ is derived from Jain's fairness index, as detailed in the following section. $\boldsymbol{w}_d$, $\boldsymbol{w}_e$, $\boldsymbol{w}_s$, and $\boldsymbol{w}_f$ are the weight parameters (with $\boldsymbol{w}_d + \boldsymbol{w}_e + \boldsymbol{w}_s + \boldsymbol{w}_f = 1$) corresponding to delay, energy consumption, task success, and fairness, respectively. The weights are normalized to sum to 1 so that the utility function can ensure consistent scale and comparability. The problem is formulated as follows:

$$\mathbf{P}: \max_{\substack{a_k^{dest},\, a_{k\to m}^{ce},\\ act_u,\, pos_{u_a}}} \sum_{h=1}^{\infty}\sum_{k=1}^{K(h)} \mathbb{U}_k(h) \tag{22a}$$

$$\text{s.t. } g_{k\to m} \le O_m^{init}, \tag{22b}$$

$$z_k^D \le C_m^{init}, \tag{22c}$$

$$v_u \le v_u^{\max}, \tag{22d}$$

$$E_u \le E_u^{\max}, \tag{22e}$$

$$\|pos_u(h) - pos_u(h-1)\| \le dist_u^{\max}. \tag{22f}$$

Here, the constraints (22b)–(22c) ensure that the task generation and data queue sizes do not exceed the initial capacities of the edge servers, while constraints (22d)–(22f) enforce limits on UAV speed, energy, and movement.

To address this problem, we decompose it into two subproblems: the first focuses on task offloading and cache placement policies to maximize total utility, while the second addresses adaptive UAV deployment to support the former. These are detailed in the next section. The overall algorithm diagram in Fig. 3 demonstrates the operation of the proposed system. The system incorporates two policy gradient (PG)-DRL algorithms, CTOP and UDMP, to manage task offloading, cache placement, and UAV deployment. A specialized FCR mechanism processes attributes like recency, frequency, and volume using FL principles to optimize data retention. The following sections provide a detailed explanation of the proposed hierarchical PPO algorithms and the FCR mechanism.

V. Hierarchical DRL-Based Task Offloading and UAV Deployment Management

In this section, we model the formulated problem as a Markov decision process (MDP), which extends a Markov chain to enable task-related decisions aimed at optimizing the utility function. We introduce the proposed UDMP-CTOP algorithm, which integrates two-timescale PPO-based decision-making framework that share state and reward information for coordinated ground-air collaboration.

A. Policy Gradient DRL

DRL learns decision policies by maximizing cumulative rewards through state-action interactions. Deep neural networks approximate value-related functions to handle high-dimensional spaces. Policy gradient methods optimize stochastic policies and underpin PPO, which adopts an actor-critic structure and stabilizes learning via a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)A_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)A_t\right)\right], \tag{23}$$

where A_t estimates the advantage function using generalized advantage estimation [45]. The ratio $r_t(\theta)$ compares the action probabilities under the current policy to the old policy:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}. \tag{24}$$

The function $\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)$ restricts $r_t(\theta)$ to prevent large updates, with ϵ as a hyperparameter. PPO also includes value function loss to improve value estimation:

$$L^{VF}(\mu) = \frac{1}{2}\mathbb{E}_t\left[\left(V_\mu(s_t) - \hat{\mathbb{R}}_t\right)^2\right], \tag{25}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

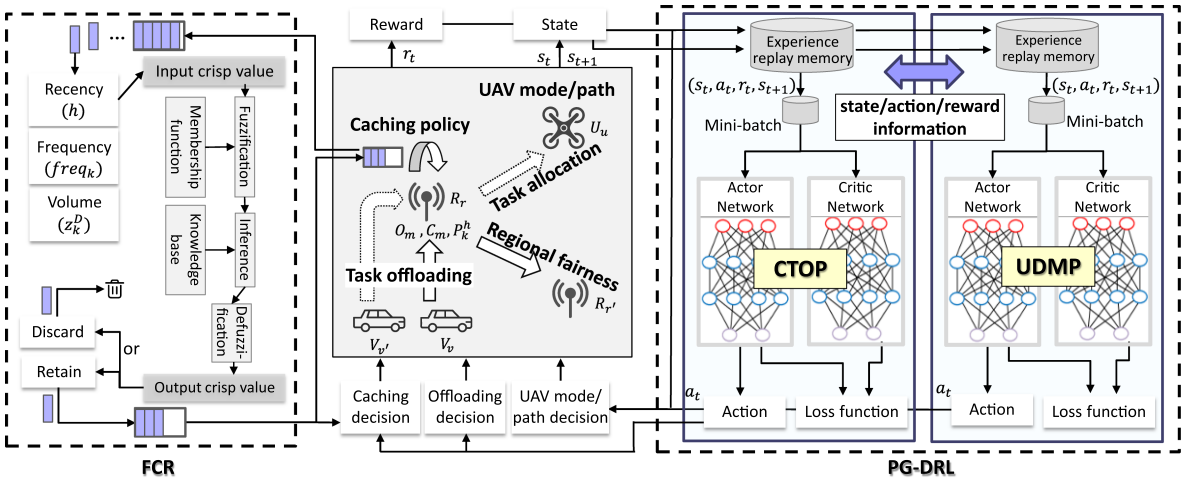


Fig. 3. Overall proposed algorithms block diagram.

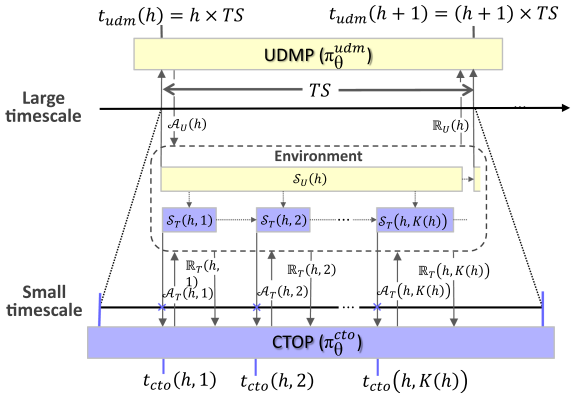


Fig. 4. Two-timescale illustration of UDMP-CTOP.

where $V_\mu(s_t)$ is the estimated value function, and $\hat{\mathbb{R}}_t$ is the target return. An entropy bonus encourages exploration by penalizing low-entropy policies:

$$L^{ENTROPY}(\theta) = \mathbb{E}_t\left[b\mathcal{H}\left(\pi_\theta\left(\cdot|s_t\right)\right)\right], \tag{26}$$

where $\mathcal{H}$ is the policy entropy, and a coefficient b controls the bonus strength.

B. Ground-Air Collaborative Task Offloading Algorithm (CTOP)

It is important to note that CTOP and UDMP algorithms operate on different timescales. In the CTOP algorithm (t_{cto}) occurs on a small timescale and is dynamic and triggered whenever a task arrives at the RSUs. On the other hand, the UDMP algorithm operates on a larger timescale (t_{udm}) being a predefined interval. UAV deployment decisions are made less frequently but with a broader and more strategic view of the status and requirements of a system. Two-timescale illustration is shown in Fig. 4.

1) State Space: The state space at h-th time slot for task k is defined as

$$\mathcal{S}_T(h,k) = \left\{P_k^h, R_{k\to r}, \mathcal{O}, \mathcal{C}, pos_{\mathcal{U}_a}\right\}, \tag{27}$$

where $P_k^h, \mathcal{O}, \mathcal{C}, pos_{\mathcal{U}_a}$ are defined in Section III. $R_{k\to r}$ denotes the nearby RSU to the vehicle. This state provides an integrated snapshot of the task, resource availability, and UAV positions.

2) Action Space: The action space defines the offloading decision and caching decision, represented as:

$$\mathcal{A}_T(h,k) = \left\{a_k^{dest}, a_{k\to m}^{ce}\right\}, \tag{28}$$

where a_k^{dest} and $a_{k\to m}^{ce}$ are defined in Section III. Once an action is taken, the Dijkstra algorithm performs task routing to find the shortest path.

3) Reward Function: The CTOP reward comprises the task successful delay, consumed energy, task retrieve, and regional fairness rewards as below:

$$\mathbb{R}_T(h,k) = \left\{\omega_1 r_k^D + \omega_2 r_k^E + \omega_3 r_k^R + \omega_4 r_k^F\right\}. \tag{29}$$

The weights ω_1, ω_2 ω_3, and ω_4 sum to 1.

The successful delay reward (r_k^D) is defined as

$$r_k^D = \begin{cases} 1 - \frac{D_k}{dt_k}, & \text{if } suc_k = 1, \\ -1, & \text{if } suc_k = 0. \end{cases} \tag{30}$$

If the task is successfully processed, a positive reward decreases as the delay approaches the tolerance limit. The reward is set to -1, if the delay exceeds the limit (i.e., $D_k > dt_k$).

The consumed energy reward (r_k^E) is the penalty for total energy consumption, which is defined as:

$$r_k^E = -\left(\frac{E_k}{E^{\max}}\right). \tag{31}$$

The task retrieve reward (r_k^R) is determined by the task frequency and caching decisions. Effective caching is crucial in minimizing the delay and energy consumption.

$$r_k^R = \begin{cases} freq_k, & \text{if } a_{k\to m}^{ce} = 1 \text{ and } freq_k \geq q(h) \text{ or} \\ & a_{k\to m}^{ce} = 0 \text{ and } freq_k < q(h), \\ -freq_k, & \text{otherwise,} \end{cases} \tag{32}$$

where $freq_k$ is defined in Section III. A positive reward is given when the caching decision aligns with the frequency threshold; otherwise, a negative reward is assigned, reflecting

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Proposed UDMP-CTOP Algorithms.

Input: $\mathcal{N}$, $\mathcal{M}$, $\mathcal{O}$, $\mathcal{C}$, $\mathcal{T}(h)$, P_k, hyperparameters
Output: Task offloading edge, caching decision, trajectory of UAVs
for each episode **do**
  Initialize the positions of UAVs
  Initialize policy networks π_θ^{cto}, π_θ^{udm} and value networks V_μ^{cto}, V_μ^{udm}
  for h-th time slot at t_{udm} **do**
    Observe state $\mathcal{S}_U(h)$
    Execute action $\mathcal{A}_U(h)$ according to π_θ^{udm}
    for h-th time slot for task k at t_{cto} **do**
      Observe state $\mathcal{S}_T(h,k)$
      Execute action $\mathcal{A}_T(h,k)$ according to π_θ^{cto}
      Calculate the task reward $\mathbb{R}_T(h,k)$ using (29)~(35)
      Collect the set of samples to experience replay buffer
      Compute loss functions and update π_θ^{cto} and V_μ^{cto}
    end
    Calculate the UAV reward $\mathbb{R}_U(h)$ using (38)~(42)
    Collect the set of samples experience replay buffer
    Compute loss functions and update π_θ^{udm} and V_μ^{udm}
  end
end

an unfavorable caching decision. The threshold function $q(h)$ varies with the memory capacity status which is defined as

$$q\left(h\right) = q_{\min} + \left(q_{\max} - q_{\min}\right)\left(1 - e^{-cC_0}\right), \tag{33}$$

where $q_{\min}$ and $q_{\max}$ are the minimum and maximum values, respectively. c is a constant scaling factor. This prioritizes high-frequency data under limited memory and permits broader storage when memory is sufficient.

The regional fairness reward $(\boldsymbol{r_k^F})$ is defined using Jain's fairness index [46] as

$$\boldsymbol{r}_k^F = F_{k,m} = \frac{\left(\sum_{m=1}^{NR} x_{k,m}\right)^2}{N_R \times \sum_{m=1}^{N_R} {x_{k,m}}^2}, \tag{34}$$

where $x_{k,m}$ denotes the accumulated service contribution of edge server m up to task k. The update rule is:

$$x_{k,m} = \sum_{k'=1}^{k} \gamma^{(k-k')} \left[suc_{k'} \frac{dt_{k'}}{D_{k'}} + \left(1 - suc_{k'}\right) \frac{dt_{k'}}{D_{k'}} \right], \tag{35}$$

where γ is a discount factor and $suc_{k'}$ indicates whether the delay requirement was satisfied for task k'. Delay is explicitly incorporated, since timely completion fundamentally determines task success. Unlike binary success/failure metrics, this formulation provides fine-grained feedback by reflecting how close a task completion time is to the deadline. After each offloading decision, the fairness reward $\boldsymbol{r}_k^F$ is recalculated based on the updated vector $\{x_{k,m}\}$. For UAV-executed tasks, the contribution is attributed to the associated RSU. This step-by-step update ensures that fairness is enforced in real time, preventing the system from persistently sacrificing certain regions for global efficiency.

C. Adaptive UAV Deployment Management Algorithm (UDMP)

1) State Space: The state space at h-th time slot for UAVs is defined as:

$$\mathcal{S}_U\left(h\right) = \left\{ \psi_{\mathcal{T}(h)}^{num}, \phi_{\mathcal{T}(h)}^{dt}, E_{\mathcal{U}_a}^{res}, \mathcal{O}, \mathcal{C}, pos_{\mathcal{U}_a}, \mathcal{S}_U\left(h-1\right) \right\}, \tag{36}$$

where $\psi_T(h)^{num}$ and $\phi_T(h)^{dt}$ denote the task density and delay tolerance distributions, capturing spatial load and urgency. $E_{U_a}^{res}$ represents the residual energy of activated UAVs. The previous state $S_U(h-1)$ is included to provide temporal context, improving policy stability.

2) Action Space: The UDMP action space at h-th time slot is expressed as

$$\mathcal{A}_U\ (h) = \{act_{\mathcal{U}}, pos_{\mathcal{U}_a}\}\ . \tag{37}$$

The activation status of all UAVs is first determined ($act_{\mathcal{U}}$), and the activated UAVs are then deployed to specific positions for the next time slot ($pos_{\mathcal{U}_a}$). If a UAV changes from active to inactive (1 to 0), it returns to the parking area. If it changes from inactive to active (0 to 1), it moves to its new position. Active UAVs (1 to 1) transition to updated positions. Once the next position is determined, the Dijkstra algorithm performs to find the shortest path for UAVs.

3) Reward Function: The reward function of UDMP is defined as follows:

$$\mathbb{R}_U\left(h\right) = \beta_1\, \boldsymbol{r}_U^G + \beta_2 \boldsymbol{r}_U^A + \beta_3 \boldsymbol{r}_U^E, \tag{38}$$

where the weights β_1, β_2, and β_3 balance the contributions of the ground performance, air utilization, and movement energy rewards, respectively.

The UAV-assisted ground performance reward $(\boldsymbol{r_U^G})$ aggregates the rewards associated with task delay, energy consumption, and regional fairness over a given time interval:

$$\boldsymbol{r}_U^G = \sum_{k=1}^{K(h)} \alpha_D \boldsymbol{r}_k^D + \alpha_E \boldsymbol{r}_k^E + \alpha_F \boldsymbol{r}_k^F, \tag{39}$$

where α_D, α_E, and α_F are weighting factors for the delay, energy, and fairness rewards from CTOP, respectively.

The air utilization reward $(\boldsymbol{r_U^A})$ quantifies the utilization of computing power and caching memory by UAVs. It is given by

$$\boldsymbol{r}_U^A = \alpha_{cp}\, \boldsymbol{r}_U^{cp} + \alpha_{ce} \boldsymbol{r}_U^{ce}, \tag{40}$$

where α_{cp} and α_{ce} the weights for computing and cache utilization, respectively. $\boldsymbol{r}_U^{cp}$ is the proportion of computing power utilized by UAVs relative to all edge servers, defined as

$$\boldsymbol{r}_U^{cp} = \frac{\sum_{m \in \mathcal{U}_a^{(ES1,va)}} \sum_{k \in \mathcal{T}_U^{cp}} g_{k \to m}}{\sum_{m \in \left\{\mathcal{R}^{(ES1)} \cup \mathcal{U}_a^{(ES1,va)}\right\}} \sum_{k \in \left\{\mathcal{T}_R^{cp} \cup \mathcal{T}_U^{cp}\right\}} g_{k \to m}}, \tag{41}$$

where $U_a^{(ES1,va)}$ denotes valid activated UAVs associated with ES1 and $R^{(ES1)}$ the RSUs with ES1; T_R^{cp} and T_U^{cp} are tasks computed on RSUs and UAVs.

Similarly, the caching utilization r_U^{ce} is defined as

$$\boldsymbol{r}_U^{ce} = \frac{\sum_{m \in \{\mathcal{U}_a^{va}\}} \sum_{k \in \{\mathcal{T}_U^{ce}\}} z_k^D a_{k \to m}^{ce}}{\sum_{m \in \{\mathcal{R} \cup \mathcal{U}_a^{va}\}} \sum_{k \in \{\mathcal{T}_R^{ce} \cup \mathcal{T}_U^{ce}\}} z_k^D a_{k \to m}^{ce}}, \tag{42}$$

where $\mathcal{U}_a^{va}$ denotes the set of valid activated UAVs for task caching, and $\mathcal{R}$ represents all RSUs capable of caching. $\mathcal{T}_R^{ce}$ and $\mathcal{T}_U^{ce}$ are the tasks cached on RSUs and UAVs, respectively.

The UAV movement energy reward $(\boldsymbol{r_U^E})$ reflects the energy consumed by UAV movements (including both deploying

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Proposed FCR Mechanism.

Input: $\mathcal{M}$, $\mathcal{C}$, C_0, $\mathcal{A}_T(h,k)$, P_k^h, membership functions, fuzzy rules, output degree
Output: Replacement decision
Initialize all edges memory status
if $a_{k\to m}^{ce}=1$ **then**
  Store the task's recency, frequency, and packet size
end if
while the memory portion of an edge C_m/C_m^{init} exceeds a threshold C_0 **do**
  for each cached data in $\mathcal{T}_R^{ce}\cup\mathcal{T}_U^{ce}$ **do**
    Convert the input parameters into crisp value
    Apply the fuzzy rules from the knowledge base
    Transform the fuzzy output into crisp value
    Execute the replacement decision based on output degree
  end
end

and returning UAVs). It is defined as

$$r_U^E = -\frac{\sum_{u=1}^{NU} E_u^{mov}}{N_U \times E_u^{\max}}. \tag{43}$$

The UDMP-CTOP algorithm is presented in Algorithm 1. The policy networks of UDMP and CTOP are represented as π_θ^{udm} and π_θ^{cto}, respectively. The value networks of UDMP and CTOP are denoted as V_μ^{udm} and V_μ^{cto}, respectively.

VI. FL-Assisted Cache Replacement Mechanism

Traditional heuristic methods such as least recently used (LRU) and least frequently used (LFU) are widely used for cache replacement. LRU evicts the least recently accessed data, while LFU removes the least frequently requested content. However, these methods perform poorly in dynamic vehicular environments, where task demands and access patterns vary unpredictably. Moreover, continuous caching decisions in CTOP may still lead to memory exhaustion under fluctuating workloads. To address this issue, we propose a cache replacement mechanism based on FL that manages memory status in real time and improves adaptability to dynamic network conditions.

A. Fuzzy Logic System

An FL system consists of four components: fuzzification, which converts crisp inputs into fuzzy values (0–1) using membership functions; a knowledge base that stores fuzzy rules and membership definitions; an inference engine that applies these rules to infer fuzzy outputs; and defuzzification, which transforms fuzzy results into crisp values using methods such as centroid, bisector, mean of maximum (MOM), or max-based approaches.

B. FCR Mechanism

The fuzzy rule system in the FCR algorithm is designed to capture three primary factors that drive caching replacement decisions: timeliness (recency), popularity (frequency), and size (volume). The domains are defined as recency $\in[-10,\ 0]$ (ten-step window), frequency $\in[0,\ 1]$, and volume $\in[0,\ 500]$ (packet-size range). To maintain compactness and interpretability, each parameter is divided into two fuzzy sets, and the membership functions map the degree of membership of each input value into corresponding linguistic categories.

TABLE III
FCR FUZZY RULES

No.	Recency	Frequency	Volume	Output
1	O	H	B	I
2	N	H	B	I
3	O	H	S	MI
4	N	H	S	MI
5	O	L	B	LI
6	N	L	B	NI
7	O	L	S	NI
8	N	L	S	M

$O, Old;\ N, New;\ L, Low;\ H, High;\ S, Small;\ B, Big$

The recency parameter has fuzzy sets $\{Old,\ New\}$. The Old set characterizes data not cached recently, with the membership decreasing linearly from full at the oldest time slots to zero at more recent time slots. The New set denotes recently cached data, with the membership increasing linearly from zero at older time slots to full at the most recent time slots. This helps optimize the caching performance by distinguishing between outdated and recent data. The frequency parameter has fuzzy sets $\{Low,\ High\}$. The Low set captures minimal data access frequency, with membership decreasing linearly from full at the lowest frequencies to zero at higher frequencies. The $High$ set denotes frequently accessed data, with membership increasing linearly from zero at low frequencies to full at higher frequencies. This differentiation assists in cache replacement by prioritizing popular data. The volume parameter has fuzzy sets $\{Small,\ Big\}$. The $Small$ set pertains to data with lower memory usage, with the membership decreasing linearly from full at the smallest volumes to zero as volume increases. The Big set includes data with larger memory usage, with the membership increasing linearly from zero at small volumes to full at larger volumes. This classification optimizes space utilization by considering the data size.

The output parameter is divided into five fuzzy sets to determine the data importance: $\{Least\ Important\ (LI),\ Not\ Important\ (NI),\ Moderate\ (M),\ Important\ (I),$ and $Most\ Important\ (MI)\}$. These sets have membership functions defining the specific ranges and transitions, reflecting the data importance. LI has the highest membership at the lowest values, while NI covers low-importance values, tapering to zero in the middle range. M peaks at moderate values, tapering off on either side. I indicates higher importance peaking in the higher range, and MI maintains full membership at the highest values. Fig. 5 presents the membership functions.

Table III summarizes the fuzzy rules and corresponding output importance levels. The consequents define an ordinal keep-priority $(LI < NI < M < I < MI)$ under three monotonic constraints: priority increases with frequency and newness, and decreases with size. These intuitive rules ensure that the FCR mechanism remains both effective and interpretable. Algorithm 2 presents the proposed FCR procedure.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

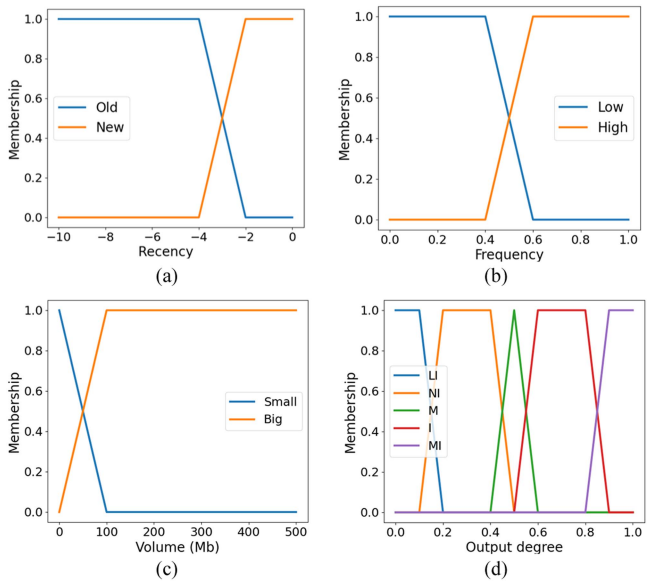


Fig. 5. FCR membership functions of (a) recency, (b) frequency, (c) volume, and (d) output degree.

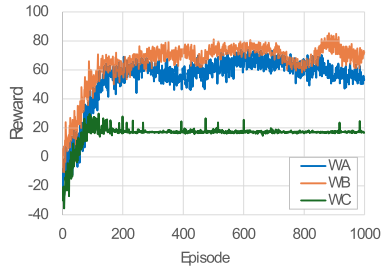


Fig. 6. nCTOP convergence across WA,WB,WC.

VII. Simulation Results

A. Complexity Analysis

The overall complexity of the proposed framework is obtained by analyzing individual components and aggregating their contributions. For CTOP in Algorithm 1, the complexity includes action selection and path routing. Under the PPO architecture, action selection requires forward propagation through a fully connected actor–critic network, with complexity $O(\sum_{l=1}^{L-1}(NN_{l-1} \times NN_l))$ according to [47], [48], where NN_l is the number of neurons in the l-th layer and L the total number of layers. The input dimension corresponds to the state space $\mathcal{S}_T(h,k)$, i.e., $O(N_R + NU_a)$. The output dimension is $O(1)$ for the critic and is $O(N_R + NU_a)$ for the actor. Path routing involves shortest-path computation among $(N_R + NU_a)$ nodes, resulting in a complexity of $O((N_R + NU_a)\log(N_R + NU_a))$. Therefore, the total CTOP complexity is:

$$Y_{ctop} = O\left(\sum_{l=1}^{L-1}(NN_{l-1} \times NN_l) + (N_R + NU_a)\log(N_R + NU_a)\right). \quad (44)$$

For UDMP, the action selection complexity follows the same PPO structure, i.e., $O(\sum_{l=1}^{L-1}(NN_{l-1} \times NN_l))$. The input corresponds to the state space $\mathcal{S}_U(h)$, which is $O(N_R + NU_a)$, while the output is $O(1)$ for the critic and $O(N_U + N_R + NU_a)$ for the actor. The routing complexity is $O(N_R \log N_R)$. Therefore, the total UDMP complexity is:

$$Y_{udmp} = O\left(\sum_{l=1}^{L-1}(NN_{l-1} \times NN_l) + N_R \log N_R\right). \quad (45)$$

For the FCR algorithm (Algorithm 2), the complexity consists of cache initialization, threshold checking, and cache replacement. Initialization requires $O(N_R + NU_a)$. When the cache exceeds the threshold, the while-loop executes at most I_{it} iterations, processes $\mathcal{T}_R^{ce} + \mathcal{T}_U^{ce}$ cached items per iteration, each evaluated using N_{rule} fuzzy rules. The resulting complexity is $O(I_{it} \times (\mathcal{T}_R^{ce} + \mathcal{T}_U^{ce}) \times N_{rule})$. Thus, the total complexity of the FCR algorithm is

$$Y_{fcr} = O\left((N_R + NU_a) + I_{it} \times (\mathcal{T}_R^{ce} + \mathcal{T}_U^{ce}) \times N_{rule}\right). \quad (46)$$

The overall complexity of proposed methods is

$$Y_{total} = Y_{ctop} + Y_{udmp} + Y_{fcr}. \quad (47)$$

B. Experimental Analysis

In this subsection, numerical simulations are conducted to evaluate the effectiveness of the proposed UDMP-CTOP algorithm with the FCR mechanism. All experiments are performed on a machine equipped with an Intel Core i5-7500 CPU, an NVIDIA GeForce RTX 3060 GPU, and Python 3.8. The algorithms are implemented in OpenAI Gym 0.25.2 with a MultiDiscrete action space composed of multiple sub-actions. A single logits vector is generated by the actor, which is partitioned per head and processed using independent softmax operations to produce categorical distributions, enabling discrete and hybrid decision-making.

The simulation environment includes 10 RSUs uniformly deployed in a 1000 m $\times$ 1000 m area, where six RSUs are assigned ES1 and the remaining four are equipped with ES2; UAVs also carry ES1. Two UAVs are initialized at (00) at the start of each episode. Task arrivals are type-balanced, with each vehicle generating one task at a random time. UDMP and CTOP adopt separate actor–critic networks with identical architectures but different depths: UDMP employs 6-layer networks, while CTOP uses 4-layer networks. Each hidden layer contains 128 neurons with ReLU activations. Both agents are trained using PPO, with hyperparameters listed in Table IV [5], [6], [30], and identical settings are used unless otherwise specified.

1) Learning Performance Evaluation: First, we evaluate the learning performance of the CTOP algorithm without UAV assistance (nCTOP). The weight sets A, B, and C listed in Table V represent different reward parameter combinations. As shown in Fig. 6, all weight settings converge at similar rates, while their final reward levels differ due to the varying emphasis in the weighted-sum reward. When delay is emphasized

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SIMULATION PARAMETERS

Description	value	Description	value
Topology area	1000 X 1000 m	Simulation duration	600 s
Number of vehicles (N_V)	200	Number of tasks	200
Number of RSUs (N_R)	10	UDMP decision interval	120 s
Number of RSUs with $ES1$, $ES2$	6, 4	Max computation resource	6 GHz
Number of UAVs (N_U)	2	Max memory capacity	3 Gb
Max vehicle velocity	20 m/s	CPU cycles per bit (ρ)	1000
Max UAV velocity (v_u^{max})	25 m/s	CPU cycles frequency (f_m)	10 GHz
UAV movement altitude	60 m	Capacitance coefficient (λ)	10^{-28}
Wireless tx power (p_i^{tx})	60 mW	Thresholds (q_{min}, q_{max})	0.1, 0.6
Wired tx rate (r_{R2R})	100 Mbps	constant scaling factor (c)	1.2
Bandwidth (B^{UL}, B^{DL})	9, 18 MHz	FCR trigger threshold (C_0)	0.5
Gaussian noise (N_0)	(- 100, 1)	FCR output degree	0.7
Channel coefficient (G_0)	(0, 1)	Batch size	256
Path loss exponent (β)	4	Discount factor	0.99
Computation request delay tolerance	0 ~ 1 s	Learning rates (actor, critic)	0.0001, 0.001
Task offloading size (z_k^O)	1 Mb	Clip	0.2
Task downloading size (z_k^D)	10 ~ 500 Mb	Lambda	0.95
Number of neurons each layer	128	CTOP NN layers (actor, critic)	4, 4
Optimizer	Adam	UDMP NN layers (actor, critic	6, 6
Entropy coefficient	0.01	Activation function	ReLu

TABLE V
WEIGHT SET DEFINITIONS

Weight definitions	Weight values
weight set A (WA)	$\omega_1 = \omega_2 = \omega_3 = \omega_4 = 0.25$
weight set B (WB)	$\omega_1 = 0.55, \omega_2 = \omega_3 = \omega_4 = 0.15$
weight set C (WC)	$\omega_1 = \omega_2 = 0.35, \omega_3 = \omega_4 = 0.15$

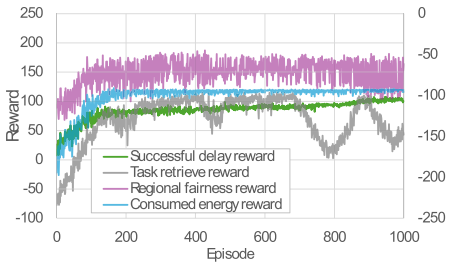


Fig. 7. nCTOP convergence of each reward under WB.

(WB), the learned policy achieves better delay performance, whereas WC places higher weights on delay and energy relative to cache retrieval and fairness, leading to a different reward scale and convergence point. These variations stem from the proportional contribution of weighted objectives to the policy gradient, resulting in distinct optimization trajectories. Although absolute reward values are not directly comparable across weight settings, all configurations converge stably to optima aligned with their respective priorities. Accordingly, WB is adopted as the representative baseline in subsequent experiments, while the framework remains adaptable through weight adjustment.

Fig. 7 shows the convergence of various rewards under the WB of nCTOP. The convergence of the consumed energy reward is plotted on a secondary axis because it is negative. All rewards successfully converge, with the task retrieve reward exhibiting varying degrees of fluctuation. This fluctuation is attributed to

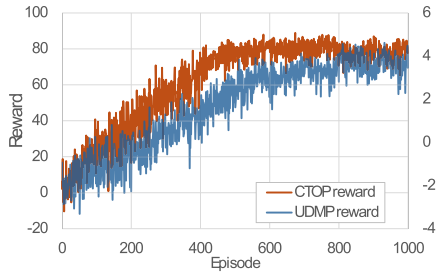


Fig. 8. UMDP-CTOP convergence performance.

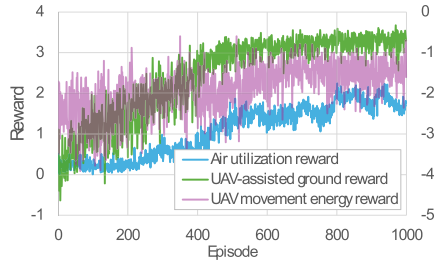


Fig. 9. UMDP convergence of each reward.

TABLE VI
SENSITIVITY ANALYSIS ON FCR MECHANISM

MF	nMAE Output Value (%)					
	Shift			*Scale*		
	5%	*10%*	*20%*	*5%*	*10%*	*20%*
Frequency-Low	5.06	6.17	8.058	0.73	1.04	1.64
Frequency-High	1.97	3.46	6.29	0.81	1.19	1.91
Recency-Old	2.38	4.13	8.24	0.78	0.94	1.27
Recency-New	1.05	1.54	3.04	0.65	0.72	0.85
Volume-Small	1.52	1.94	2.35	0.64	0.65	0.67
Volume-Big	0.84	1.09	2.28	0.66	0.67	0.71
Output-LI	1.77	2.9	4.97	0.65	0.67	0.72
Output-NI	1.29	1.98	3.52	0.65	0.68	0.74
Output-M	0.69	0.75	0.92	0.66	0.69	0.76
Output-I	2.42	4.39	7.3	0.64	0.66	0.7
Output-MI	0.57	0.76	7.04	0.65	0.67	0.71

the dynamic cache replacement process in FCR, where changes in the remaining memory affect the caching decisions and the associated reward.

Fig. 8 shows the learning convergence effects of UDMP-CTOP. For comparison, the UDMP reward is displayed on a secondary axis. In the early learning stages, the varying positions of the UAVs introduce uncertainty in the CTOP state information, leading to relatively slower learning. By the end, both sides achieve successful convergence, with the highest CTOP reward exceeding the maximum value observed in Fig. 5. The convergence of various rewards in the UDMP reward is presented in Fig. 9. Similarly, the UAV movement energy reward is plotted on a secondary axis, and all three rewards successfully converge.

Table VI reports the sensitivity analysis of the proposed FCR mechanism. A one-factor-at-a-time approach is applied to all input and output membership functions (MFs), considering shift (boundary drift) and scale (width change) perturbations of $\pm 5\%$, $\pm 10\%$, and $\pm 20\%$. For each case, outputs are recomputed under identical inputs, and the normalized mean absolute error

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

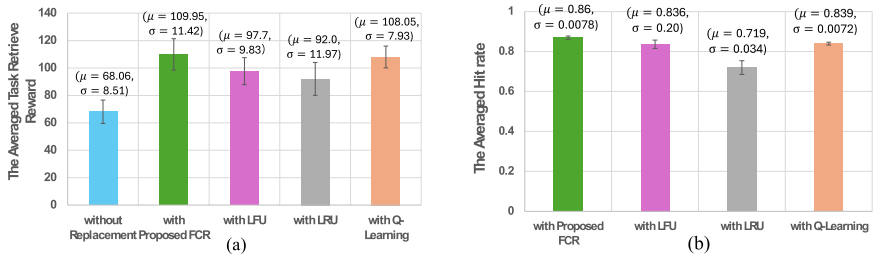


Fig. 10. FCR mechanism comparison on (a) the averaged task retrieve reward, and (b) the hit rate.

(nMAE, %) is calculated as nMAE = MAE/(output range) × 100. Scale perturbations have consistently minor impacts across all MFs (typically 0.6–1.9%), indicating low sensitivity to MF slopes and widths. In contrast, boundary shifts dominate the sensitivity, with Frequency-Low and Recency-Old being the most influential cut-points, reaching high single-digit deviations under ±20% perturbations. Other inputs show smaller deviations (around 4%), while output-side shifts remain moderate (generally below 5%). Notably, within the practical ±10% range, nearly all deviations fall between 0.5–4%, demonstrating the robustness of the FCR and indicating that sensitivity is concentrated in a few intuitive boundary locations.

To evaluate the effectiveness of the FCR mechanism, we compare it with two rule-based cache replacement strategies, LRU and LFU [49], [50], and a Q-learning-based method adapted from [52]. For fairness, Q-learning uses the same state features (recency, frequency, and volume) and a binary action space (replace/keep) as FCR. Its reward function is defined as

$$\mathrm{R} = \begin{cases} b_1 \times freq_k + b_2 \times \frac{1}{1-h} - b_3 \times z_k^D, \ a = 0, \\ -b_1 \times freq_k - b_2 \times \frac{1}{1-h} + b_3 \times z_k^D, a = 1, \end{cases} \tag{48}$$

with a learning rate of 0.001, a discount factor of 0.9, and coefficients $b_1 = b_2 = 1$, $b_3 = \frac{1}{500}$ for normalization. As shown in Fig. 10(a), no-replacement case yields the lowest average task retrieval reward of 68.06 ($\sigma = 8.51$). Q-learning performs strongly with an average task-retrieval reward of 108.05 ($\sigma = 7.93$), while FCR achieves the highest mean of 109.95 ($\sigma = 11.42$). Both clearly outperform LRU and LFU, with Q-learning also exhibiting the lowest variance. For the average hit rate as shown in Fig. 10(b), LRU yields the lowest hit rate (< 0.8), while LFU and Q-learning achieve 0.836 and 0.839, respectively. FCR is observed at the highest as 0.86 ($\sigma \approx 0.0078$) confirming superior steady-state effectiveness. Although Q-learning demonstrates competitive performance with reduced variance, it introduces training overhead and memory cost for Q-table updates. In contrast, the proposed FCR is lightweight, interpretable, and readily deployable, confirming its advantage in both performance and practicality.

2) Performance Evaluation With Comparative Approaches: To evaluate the performance of proposed UDMP-CTOP, four other schemes are chosen for comparison as follows:

- Greedy-based Task Offloading (GbTO): This heuristic approach selects the offloading edge greedily based on the closest distance, without request routing among edges.
- DDQN: In this scheme, a double deep Q-network is adapted to allocate dynamic resources for task offloading. The task and edge environment are replicated using the same total resources as in our setup. All other settings follow the original work [36].
- Non-UAV CTOP (nCTOP): In this scheme, no UAV assistance is provided to the environment; the proposed CTOP is performed on the ground environment.
- Demand-based Fixed Trajectory CTOP (DbFT-CTOP): In this scheme, fixed UAV trajectories are used for all UAVs, assuming that the future task demands across the entire environment are known in advance—an unrealistic assumption in real-world scenarios. This setup serves as a benchmark.

Fig. 11 illustrates the average task success rate under different resource scenarios. As shown in Fig. 11(a), the success rate increases with uplink bandwidth from 1 MHz to 17 MHz. UDMP-CTOP and DbFT-CTOP reach about 90% in the high-bandwidth regime, while nCTOP peaks at approximately 88%; DDQN and GbTO perform worse, with GbTO falling below 80%. In Fig. 11(b), increasing computation resources slightly improves CTOP-based methods, whereas DDQN and GbTO remain largely unchanged and below 80%. In Fig. 11(c), the success rate drops sharply as request data size grows, and performance across methods becomes similar. UDMP-CTOP consistently achieves the highest success rate, marginally outperforming DbFT-CTOP. Unless otherwise stated, 9 MHz bandwidth, 6 GHz computation capacity, and 1 MB data size are used as default values.

Fig. 12 shows the average task energy consumption, where higher curves (closer to zero) indicate lower energy usage. In Fig. 12(a), increasing bandwidth reduces communication energy for all methods. Fig. 12(b) shows that additional computation resources lower energy consumption mainly for nCTOP and DDQN, while UDMP-CTOP and DbFT-CTOP change little, reflecting reduced dependence on ground computation due to UAV assistance. In Fig. 12(c), larger data sizes increase communication energy, most significantly for GbTO. Overall, GbTO has the highest energy consumption, whereas UDMP-CTOP and DbFT-CTOP achieve the best energy efficiency, followed by DDQN and nCTOP.

Fig. 13 presents regional fairness under varying resources, where lower values indicate greater imbalance. In Fig. 13(a), higher bandwidth concentrates traffic on well-connected edges, reducing fairness. Fig. 13(b) shows that increased computation

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

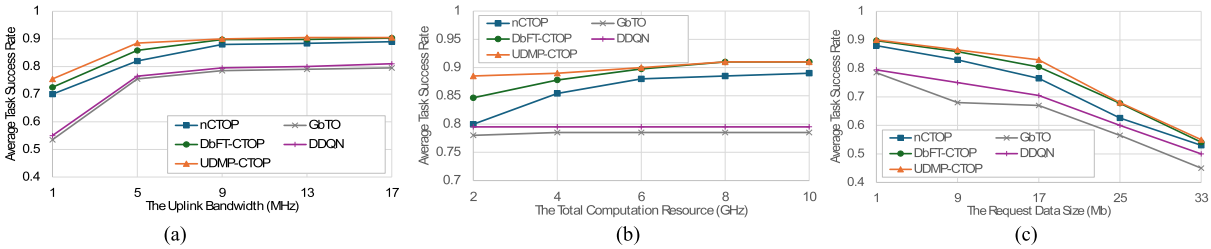


Fig. 11. Average task success rate across (a) different uplink bandwidths, (b) different total computation resource, and (c) different request data size.

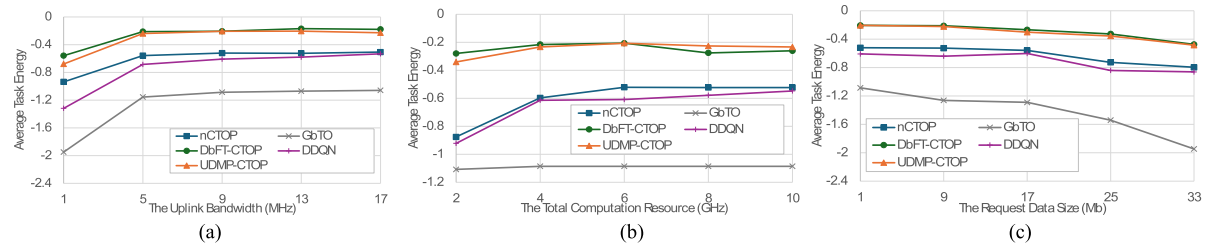


Fig. 12. Average task energy consumption across (a) different uplink bandwidths, (b) different total computation resource, and (c) different request data size.

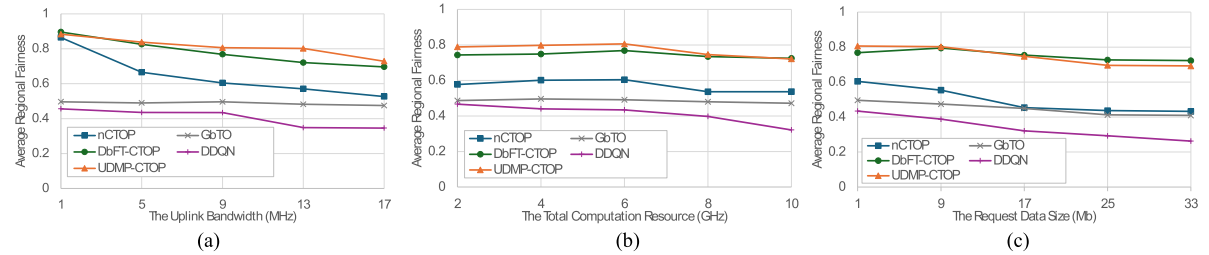


Fig. 13. The average regional fairness across (a) different uplink bandwidths, (b) different total computation resource, and (c) different request data size.

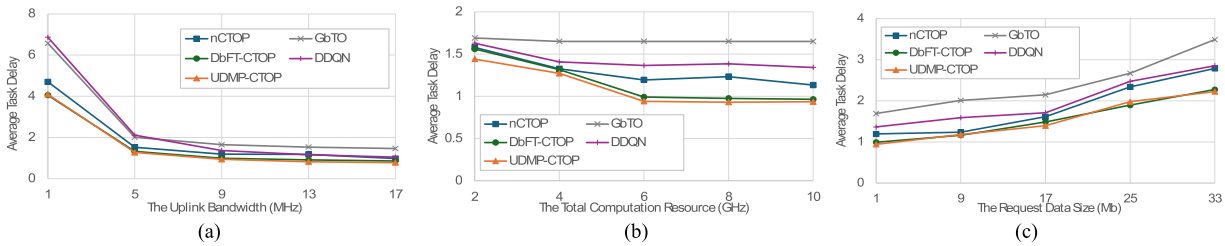


Fig. 14. The average task delay across (a) different uplink bandwidths, (b) different total computation resource, and (c) different request data size.

capacity amplifies load concentration on high-capacity edges, while UAV-assisted methods alleviate this effect. In Fig. 13(c), larger data sizes further exacerbate spatial imbalance. Across all scenarios, UDMP-CTOP and DbFT-CTOP maintain fairness above approximately 0.7, whereas nCTOP, DDQN, and GbTO exhibit notably lower levels.

Fig. 14 depicts the average task delay. Delay decreases with increasing bandwidth and computation resources in Fig. 14(a) and (b), with UDMP-CTOP and DbFT-CTOP achieving delays below 1 s, while GbTO remains above 1.6 s. In Fig. 14(c), delay increases with data size, but UAV-assisted methods show the slowest growth, maintaining superior performance.

Finally, Fig. 15 compares total utility under different resource scenarios. UDMP-CTOP consistently achieves the highest utility, slightly outperforming DbFT-CTOP. nCTOP ranks above DDQN but below UAV-assisted methods, while GbTO shows the lowest utility across all scenarios.

To explicitly evaluate the benefit of the proposed fairness mechanism, we compare UDMP-CTOP with the fairness-aware offloading baseline MSE-PPO [51]. MSE-PPO adopts an actor–critic parallel architecture and repositions UAVs every 30 s to maximize offloading while enforcing fair visitation of ground devices, whereas UDMP-CTOP uses a longer 120 s repositioning interval and directly embeds fairness into the reward function. As shown in Fig. 16, UDMP-CTOP consistently achieves

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

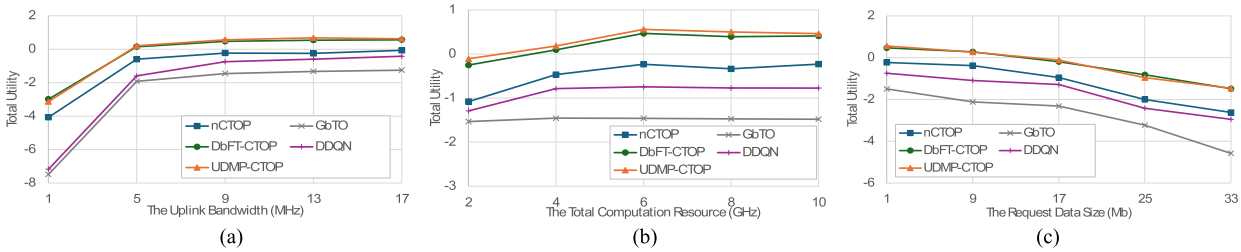


Fig. 15. The average total utility across (a) different uplink bandwidths, (b) different total computation resource, and (c) different request data size.

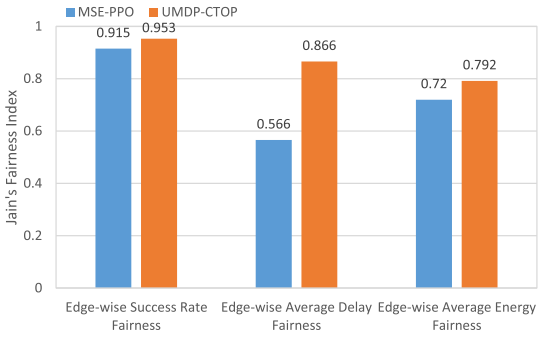


Fig. 16. Edge-wise fairness performance comparison.

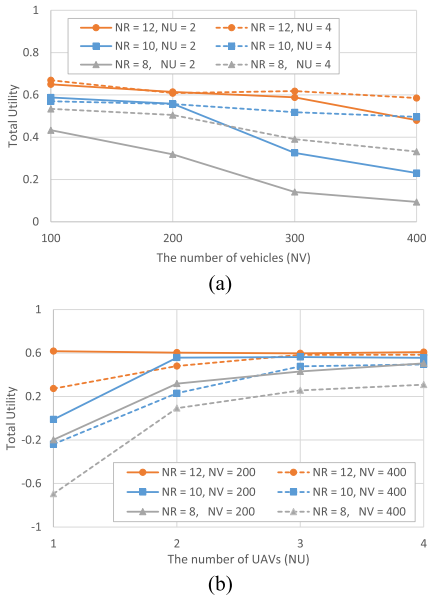


Fig. 17. The average total utility across (a) the number of vehicles, and (b) the number of UAVs.

higher fairness across success rate, delay, and energy. Specifically, success rate fairness increases from 0.915 to 0.953, delay fairness from 0.566 to 0.866, and energy fairness from 0.720 to 0.792. These results indicate that although MSE-PPO benefits from frequent UAV repositioning, it fails to alleviate delay imbalance at overloaded nodes, while UDMP–CTOP delivers more balanced performance across heterogeneous edge servers with higher overall utility.

Fig. 17(a) shows the variation of total utility as the number of vehicles N_V increases from 100 to 400 within a fixed area. Each

TABLE VII
UDMP-CTOP PERFORMANCE UNDER SMALL, MODERATE, AND LARGE-SCALE DEPLOYMENT

Scales		Small ($N_R = 8, N_U = 2, N_V = 200$)	Moderate ($N_R = 12, N_U = 4, N_V = 400$)	Large ($N_R = 20, N_U = 6, N_V = 800$)
Performance	Utility	0.319	0.585	0.485
	Rate	0.93	0.912	0.962
	Fairness	0.861	0.919	0.712
	Delay	1.162	1.024	0.985
	Energy	−0.310	−0.221	−0.204
Model Parameters	CTOP	0.141 million	0.145 million	0.152 million
	UDMP	0.219 million	0.236 million	0.266 million

curve corresponds to a specific configuration of RSUs (N_R) and UAVs (N_U), where solid and dotted lines denote $N_U = 2$ and $N_U = 4$, respectively. Utility declines more rapidly for $N_U = 2$, indicating that additional UAVs help sustain performance under higher task density. Scenarios with more RSUs (e.g., $N_R = 12$) exhibit smaller degradation due to higher ground processing capacity, whereas configurations with fewer RSUs experience steeper drops. Fig. 17(b) illustrates utility as N_U increases from 1 to 4 for $N_V = 200$ and $N_V = 400$. Overall, increasing N_U improves utility by relieving RSU congestion. When ground resources are sufficient ($N_R = 12$, $N_V = 200$), additional UAVs yield marginal gains, while in high-demand, resource-limited scenarios ($N_R = 8$, $N_V = 400$), UAV assistance becomes critical. As N_U grows, utility improves across all settings, with performance gaps narrowing at $N_U = 4$.

Table VII summarizes the performance of the proposed UDMP–CTOP framework under small, moderate, and large-scale deployments. In the small-scale scenario, the limited number of edges and sparse spatial distribution force tasks to traverse longer distances, resulting in a slightly higher end-to-end delay (1.162 s), although the success rate remains acceptable at 0.93. In the moderate-scale scenario, system performance is the most balanced, achieving the highest overall utility (0.585). Delay decreases to 1.024 s, while success rate (0.912) and fairness (0.919) remain stable, indicating that moderate resource availability with manageable heterogeneity yields robust performance. In the large-scale scenario, the abundance of edges further increases the success rate to 0.962, as tasks almost always find feasible offloading targets. However, fairness degrades to 0.712 due to heterogeneous capacities and uneven edge distributions, where some regions become overloaded while others remain underutilized. From a computational perspective, the scalability of CTOP and UDMP is analyzed. For CTOP, the state and action spaces are $|\ S_T(h,k)\ | = 10 + 2N_R + 3N_U$ and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
UAV FLIGHT ENERGY CONSUMPTION COMPARISON

Average UAV movement energy (KJ)	$1TS$	$2TS$	$3TS$	$4TS$	$5TS$	Total
UDMP	36	98.889	61.287	91.992	72	360.168
DbFT	81.8	111.2	93.087	102.262	92.462	480.811

TABLE IX
IMPACT OF WIND DISTURBANCES ON UDMP-CTOP

UDMP Reward	No wind	Light breeze=2m/s	Moderate breeze=6m/s
Ground performance reward	2.23±0.25	1.97±0.21	1.41±0.28
UAV utilization reward	0.29±0.04	0.26±0.02	0.17±0.14
UAV movement energy reward	-2.75±0.04	-2.77±0.05	-2.77±0.08

$| A_T(h,k) | = N_R + N_U + 2$, respectively. Using a 4-layer fully connected network with 128 neurons per layer, CTOP contains approximately 0.15 million parameters. For UDMP, the state and action spaces scale as $| S_U(h) | = (4N_R + 4N_U) \times 2$ and $| A_U(h) | = (N_R + 2) \times N_U$, with a deeper 6-layer network of 128 neurons per layer. As a result, UDMP's parameter size increases by 21.5% from small to large scale, compared with 7.8% for CTOP, indicating higher computational growth for UDMP.

DbFT-CTOP and UDMP-CTOP both achieve strong performance in supporting ground task offloading, but differ in UAV movement energy consumption. As shown in Table VIII, DbFT incurs higher overall flight energy without UAV return, whereas UDMP dynamically manages UAV support and reduces energy consumption by approximately 25%.

Table IX evaluates the impact of wind disturbances on the UDMP-CTOP framework under three conditions: no wind, light breeze (2 m/s), and moderate breeze (6 m/s). The flight altitude follows the default setting, with wind directions uniformly sampled over $[0°, 360°]$. Results are reported as mean ± standard deviation across multiple seeds for the three UDMP reward components. Compared with the no-wind baseline, ground performance decreases by 11.7% (2.23→1.97) at 2 m/s and by 36.8% (2.23→1.41) at 6 m/s, while air utilization drops by 10.3% (0.29→0.26) and 41.4% (0.29→0.17), respectively. These results indicate that stronger winds significantly reduce aerial serviceability, shifting load back to ground RSUs and degrading overall ground performance. In contrast, under low mobility uncertainty, UAV assistance yields clear performance gains. Energy consumption remains nearly unchanged across conditions, likely due to offsetting headwind and tailwind effects and reduced aerial utilization that shortens UAV flight time. Overall, Table IX shows that UAV assistance is most effective under mild wind conditions, while the system adaptively reduces aerial usage as wind intensity increases.

Fig. 18 shows the trajectories of UAVs using UDMP-CTOP in each TS. The blue lines represent the trajectory of the first UAV, while the purple lines represent the trajectory of the second UAV. The circles indicate the RSU positions, with the circle size representing task demand—the larger the circle, the higher the number of tasks. The dark blue circles denote RSUs with $ES1$, while the grey circles denote RSU with $ES2$. The deactivation area indicates in which TS the UAVs are not activated. The first UAV departs at $2TS$ and travels to R_6, passes R_{10}, and eventually

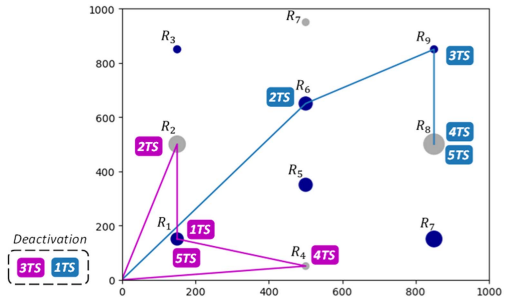


Fig. 18. Trajectories of UAVs based on UDMP-CTOP.

hovers over R_9. The second UAV departs at $1TS$, travels to R_1, passes R_2, and returns to the deactivation area at $2TS$ before flying to R_4, and R_1 in the next TS. This illustrates the cooperative coordination between the two UAVs and their autonomous return based on the ground demand to minimize fuel costs.

VIII. CONCLUSION

This paper proposes an adaptive ground–air collaboration framework that integrates RSUs and UAVs for fairness-aware task offloading in vehicular edge computing. We develop UDMP-CTOP, a two-timescale hierarchical PPO approach, where UDMP controls scalable UAV deployment at a coarse timescale and CTOP performs fine-grained task offloading and cache placement at a finer timescale. To address limited edge storage, a fuzzy logic–based cache replacement (FCR) mechanism is introduced, prioritizing data based on recency, frequency, and volume. Extensive simulations demonstrate that the proposed framework significantly improves task latency, energy consumption, task success rate, and regional fairness compared with existing methods. While the centralized design achieves strong performance, its scalability may be constrained in large-scale multi-agent settings due to increased complexity and training overhead. Future work will investigate distributed learning paradigms, such as federated learning, to enhance scalability and adaptability. Currently, no unified end-to-end standard exists for task offloading across RSUs and UAVs. Existing standards, including ETSI MEC federation and 3GPP edge features, provide partial inter-edge signaling support, while practical deployments often rely on implementation-specific control channels. Accordingly, we assume a pre-established control fabric and focus on data-plane performance to enable reproducible evaluation of the proposed algorithms. In practice, signaling overhead can be reduced through task aggregation, persistent control channels, and MEC orchestrators. A comprehensive analysis of signaling costs using standardized MEC APIs will be explored in future work.

REFERENCES

[1] S.-J. Yoo and S.-H. Choi, "Indoor AR navigation and emergency evacuation system based on machine learning and IoT technologies," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 20853–20868, Nov. 2022.
[2] J.-H. Park, Q. Yang, and S.-J. Yoo, "RHRA-DRL: RSU-assisted hybrid road-aware routing using distributed reinforcement learning in Internet of Vehicles," *IEEE Access*, vol. 12, pp. 25385–25396, 2024.
[3] Q. Yang and S.-J. Yoo, "Optimizing intersection navigation with multishot update mechanism: A real-time distributed reinforcement learning for ground and aerial moving objects," *IEEE Internet Things J.*, vol. 12, no. 19, pp. 40650–40665, Oct. 2025.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

[4] A. Paul, K. Singh, C.-P. Li, O. A. Dobre, and T. Q. Duong, "Digital twin-aided vehicular edge network: A large-scale model optimization by quantum-DRL," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 2156–2173, Feb. 2025.

[5] C. Liu, Y. Zhong, R. Wu, S. Ren, S. Du, and B. Guo, "Deep reinforcement learning based 3D-trajectory design and task offloading in UAV-enabled MEC system," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 3185–3195, Feb. 2025.

[6] H. Guo, J. Zhang, and J. Liu, "FiWi-enhanced vehicular edge computing networks: Collaborative task offloading," *IEEE Veh. Technol. Mag.*, vol. 14, no. 1, pp. 45–53, Mar. 2019.

[7] Q. Yang and S.-J. Yoo, "Hierarchical reinforcement learning-based routing algorithm with grouped RSU in urban VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 10131–10146, Aug. 2024.

[8] A. Beishenalieva and S.-J. Yoo, "Multiobjective 3-D UAV movement planning in wireless sensor networks using bioinspired swarm intelligence," *IEEE Internet Things J.*, vol. 10, no. 9, pp. 8096–8110, May 2023.

[9] Z. Liao, C. Yuan, B. Zheng, and X. Tang, "An adaptive deployment scheme of unmanned aerial vehicles in dynamic vehicle networking for complete offloading," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23509–23520, Jul. 2024.

[10] Z. Kuang, H. Wang, J. Li, and F. Hou, "Utility-aware UAV deployment and task offloading in multi-UAV edge computing networks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14755–14770, Apr. 2024.

[11] Z. Zhang, X. Wei, C.-H. Lung, and Y. Zhao, "iCache: An intelligent caching scheme for dynamic network environments in ICN-based IoT networks," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1787–1799, Jan. 2023.

[12] S. Zhu, X. Tian, H. Chen, H. Zhu, and R. Qiao, "Edge collaborative caching solution based on improved NSGA II algorithm in Internet of Vehicles," *Comput. Netw.*, vol. 244, 2024, Art. no. 110307.

[13] Y. Liu, C. Yang, X. Chen, and F. Wu, "Joint hybrid caching and replacement scheme for UAV-assisted vehicular edge computing networks," *IEEE Trans. Intell. Veh.*, vol. 9, no. 1, pp. 866–878, Jan. 2024.

[14] G.-S. Nam, Q. Yang, and S.-J. Yoo, "Joint optimization of vehicle and pedestrian traffic signals using multi-objective deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 1, pp. 501–520, Jan. 2026.

[15] Q. Yang and S. J. Yoo, "Multi-objective task offloading optimization using deep reinforcement learning with resource distribution clustering," *ICT Exp.*, vol. 11, no. 4, pp. 734–742, 2025.

[16] A. Beishenalieva and S.-J. Yoo, "Secrecy-aware UAV path planning and offloading strategy optimization using deep reinforcement learning and particle swarm optimization," *IEEE Trans. Intell. Transp. Syst.*, early access, Nov. 19, 2025, doi: 10.1109/TITS.2025.3631889.

[17] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.

[18] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.

[19] Q. Luo, C. Li, T. H. Luan, W. Shi, and W. Wu, "Self-learning based computation offloading for internet of vehicles: Model and algorithm," *IEEE Trans. Wireless Commun.*, vol. 20, no. 9, pp. 5913–5925, Sep. 2021.

[20] Y. K. Tun, T. N. Dang, K. Kim, M. Alsenwi, W. Saad, and C. S. Hong, "Collaboration in the sky: A distributed framework for task offloading and resource allocation in multi-access edge computing," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 24221–24235, Dec. 2022.

[21] Y. Xu, T. Zhang, J. Loo, D. Yang, and L. Xiao, "Completion time minimization for UAV-assisted mobile-edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12253–12259, Nov. 2021.

[22] C. Zhan, H. Hu, X. Sui, Z. Liu, and D. Niyato, "Completion time and energy optimization in the UAV-enabled mobile-edge computing system," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7808–7822, Aug. 2020.

[23] H. Guo and J. Liu, "UAV-enhanced intelligent offloading for internet of Things at the edge," *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2737–2746, Apr. 2020.

[24] Y. Liu, K. Xiong, Q. Ni, P. Fan, and K. B. Letaief, "UAV-assisted wireless powered cooperative mobile edge computing: Joint offloading, CPU control, and trajectory optimization," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2777–2790, Apr. 2020.

[25] X. Dai, Z. Xiao, H. Jiang, and J. C. S. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.

[26] M. A. Mirza et al., "DRL-assisted delay optimized task offloading in automotive-Industry 5.0 based VECNs," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 6, 2023, Art. no. 101512.

[27] X. Deng et al., "Task offloading in internet of vehicles: A DRL-based approach with representation learning for DAG scheduling," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5045–5060, Jun. 2025.

[28] J. Xue, F. Shao, T. Zhang, G. Tian, and H. Jiang, "VEC collaborative task offloading and resource allocation based on deep reinforcement learning under parking assistance," *Wireless Pers. Commun.*, vol. 136, no. 1, pp. 321–345, 2024.

[29] Z. Wu, Z. Jia, X. Pang, and S. Zhao, "Deep reinforcement learning-based task offloading and load balancing for vehicular edge computing," *Electronics*, vol. 13, no. 8, 2024, Art. no. 1511.

[30] Z. Gao, J. Fu, Z. Jing, Y. Dai, and L. Yang, "MOIPC-MAAC: Communication-assisted multi-objective MARL for trajectory planning and task offloading in multi-UAV assisted MEC," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18483–18502, May 2024.

[31] W. Lee and T. Kim, "Multiagent reinforcement learning in controlling offloading ratio and trajectory for multi-UAV mobile-edge computing," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 3417–3429, Jan. 2024.

[32] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.

[33] C. Li et al., "Deep reinforcement learning-based mining task offloading scheme for intelligent connected vehicles in UAV-aided MEC," *ACM Trans. Des. Automat. Electron. Syst.*, vol. 29, no. 3, pp. 1–29, 2024.

[34] J. Yan, X. Zhao, and Z. Li, "Deep reinforcement learning based computation offloading in UAV-assisted vehicular edge computing networks," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19882–19897, Jun. 2024.

[35] C. Wang, C. Liang, F. R. Yu, Q. Chen, and L. Tang, "Computation offloading and resource allocation in wireless cellular networks with mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 16, no. 8, pp. 4924–4938, Aug. 2017.

[36] L. Wang and G. Zhang, "Joint service caching, resource allocation and computation offloading in three-tier cooperative mobile edge computing system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3343–3353, Nov./Dec. 2023.

[37] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.

[38] Z. Ning et al., "Intelligent edge computing in internet of vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, Apr. 2021.

[39] L. Liu, X. Yuan, N. Zhang, D. Chen, K. Yu, and A. Taherkordi, "Joint computation offloading and data caching in multi-access edge computing enabled internet of vehicles," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 14939–14954, Nov. 2023.

[40] A. S. Ghorab, R. S. Rasheed, and H. M. Hamad, "Joint microservices caching and task offloading framework in VEC based on deep reinforcement learning," *Int. J. Interactive Mobile Technol.*, vol. 17, no. 8, pp. 78–99, 2023.

[41] Y. Wang et al., "A survey on deploying mobile deep learning applications: A systemic and technical perspective," *Digit. Commun. Netw.*, vol. 8, no. 1, pp. 1–17, 2022.

[42] B. Chen, L. Liu, M. Sun, and H. Ma, "IoTCache: Toward data-driven network caching for Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10064–10076, Dec. 2019.

[43] X. Zhao and Q. Zhu, "Mobility-aware and interest-predicted caching strategy based on IoT data freshness in D2D networks," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 6024–6038, Apr. 2021.

[44] A. Beishenalieva and S.-J. Yoo, "UAV path planning for data gathering in wireless sensor networks: Spatial and temporal substate-based Q-learning," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9572–9586, Mar. 2024.

[45] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–14.

[46] G. Zhang, F. Shen, Y. Yang, H. Qian, and W. Yao, "Fair task offloading among fog nodes in fog computing networks," in *Proc. IEEE Int. Conf. Commun.*, Kansas City, MO, USA, 2018, pp. 1–6.

[47] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.

[48] F. Song et al., "AoI and energy tradeoff for aerial-ground collaborative MEC: A multi-objective learning approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11278–11294, Dec. 2024.
[49] S. Kumar and P. K. Singh, "An overview of modern cache memory and performance analysis of replacement policies," in *Proc. 2016 IEEE Int. Conf. Eng. Technol.*, 2016, pp. 210–214.
[50] Z. Wang, J. Hu, G. Min, Z. Zhao, and Z. Wang, "Agile cache replacement in edge computing via offline-online deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 4, pp. 663–674, Apr. 2024.
[51] W. Li, S. Li, H. Shi, W. Yan, and Y. Zhou, "UAV-enabled fair offloading for Mec networks: A DRL approach based on actor-critic parallel architecture," *Appl. Intell.*, vol. 54, no. 4, pp. 3529–3546, 2024.
[52] Z. Zhang, T. Cao, X. Wang, H. Xiao, and J. Guan, "VC-PPQ: Privacy-preserving Q-learning based video caching optimization in mobile edge networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 6, pp. 4129–4144, Nov./Dec. 2022.



Qin Yang (Graduate Student Member, IEEE) received the B.E. degree in communication and information engineering from the Chongqing University of Posts and Telecommunications, Chongqing, China, in 2016, and the M.S. degree in electrical and computer engineering from Inha University, Incheon, South Korea, in 2018, where she is currently working toward the Ph.D. degree with Multimedia Network Laboratory. Her research interests include machine learning, reinforcement learning, wireless sensor networks, vehicular networks, and the Internet of Things.



Sang-Jo Yoo (Member, IEEE) received the B.S. degree in electronic communication engineering from Hanyang University, Seoul, South Korea, in 1988, and the M.S. and Ph.D. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea, in 1990 and 2000, respectively. From 1990 to 2001, he was a Member of the Technical Staff with the KT Research and Development Group. From 1994 to 1995, and from 2007 to 2008, he was a Guest Researcher with the National Institute of Standards and Technology, USA. Since 2001, he has been with Inha University, Incheon, South Korea, where he is currenlty a Professor with the Electrical and Computer Engineering Department. His research interests include machine learning, cognitive radio networks, vehicular networks, wireless sensor networks, and the Internet of Things.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:10:53 UTC from IEEE Xplore. Restrictions apply.